4. 실험 설계와 결과

본 장에서는 지금까지 제안한 TDA 기반 음악 생성 파이프라인의 성능을 정량적으로 평가한다. 세 가지 유형의 실험을 수행하였다.

1. **Distance function 비교** — frequency(기본), Tonnetz, voice-leading, DFT 네 종류의 거리 함수에 대해 동일 파이프라인을 적용하고 생성 품질을 비교.
2. **Cycle subset ablation** — 최적 거리 함수(Tonnetz)에서 cycle 수를 $K = 10, 17, 46$으로 변화시켜 cycle 수의 효과를 분리.
3. **통계적 유의성** — 각 설정에서 Algorithm 1을 $N = 20$회 독립 반복 실행하여 mean ± std를 보고.

모든 실험은 동일한 chord height 패턴 (32-element module × 33 = 1,056 timepoints), 동일 random seed 체계($s = c + i,\ i = 0, \ldots, 19$, 설정별 상수 c 사용)로 수행되었다. 실험 러너는 `tda_pipeline/run_step3_experiments.py`이며, 모든 trial의 상세 기록(mean, std, min, max 포함)은 `tda_pipeline/docs/step3_data/step3_results.json`에 저장되어 있다.

평가 지표

Jensen-Shannon Divergence (주 지표). 생성곡과 원곡의 pitch 빈도 분포 간 JS divergence를 주 지표로 사용한다 (2.6절 정의). 값이 낮을수록 두 곡의 음 사용 분포가 유사하며, 이론상 최댓값은 $\log 2 \approx 0.693$이다.

Note Coverage. 원곡에 존재하는 고유 (pitch, duration) 쌍 중, 생성곡에 한 번 이상 등장하는 쌍의 비율. 1.00이면 모든 note가 최소 한 번 이상 사용된 것이다.

보조 지표. Pitch count (생성곡의 고유 pitch 수), 생성 소요 시간 (초), KL divergence.

거리 함수 구현 참고

본 장의 네 가지 거리 함수는 모두 `tda_pipeline/musical_metrics.py` 파일 하나에 정의되어 있다. 사용 패턴은 다음과 같다.

함수	역할	입력	출력
`tonnetz_distance(pc1, pc2)`	두 pitch class 간 Tonnetz 격자 거리	`pc1, pc2` $\in \{0, \ldots, 11\}$	정수 (최단 경로 길이)
`tonnetz_note_distance(n1, n2)`	두 note 간 거리 (옥타브/duration 보정 포함)	`(pitch, duration)` 튜플 × 2	실수
`voice_leading_note_distance(n1, n2)`	두 note 간 semitone 차이 기반 거리	`(pitch, duration)` 튜플 × 2	실수
`dft_note_distance(n1, n2)`	pitch class DFT 계수 간 Euclidean 거리	`(pitch, duration)` 튜플 × 2	실수
`compute_note_distance_matrix(notes_label, metric)`	전체 note 집합의 거리 행렬 생성	`{(pitch,dur): label}` 딕셔너리, metric 이름	$N \times N$ numpy 행렬

두 note 간 확장 — 옥타브와 duration 보정. 위의 거리 함수들은 원래 pitch class(mod 12)만 고려하므로 옥타브와 duration 정보가 손실된다. 본 연구에서 note는 (pitch, duration) 쌍으로 정의되므로, 세 거리 함수 모두에 다음 두 항을 추가한다.

$$d(n_1, n_2) = d_{\text{base}}(p_1, p_2) + w_o \cdot |o_1 - o_2| + w_d \cdot \frac{|d_1 - d_2|}{\max(d_1, d_2)}$$

여기서 d_{base}는 Tonnetz / voice-leading / DFT 중 하나, $o_i = \lfloor p_i / 12 \rfloor$는 옥타브 번호, d_i는 duration, $w_o = 0.5$, $w_d = 0.3$이다.

각 항의 설계 근거:

- **옥타브 항** $w_o|o_1 - o_2|$: 같은 pitch class(예: C4와 C5)라도 옥타브가 다르면 음악적으로 다른 역할을 한다. $w_o = 0.5$는 "1 옥타브 차이가 Tonnetz 거리 2에 해당하는 영향"을 주도록 설정한 값이다.
- **Duration 항** $w_d|d_1 - d_2|/\max(d_1, d_2)$: 분자를 $\max$로 정규화하여 $[0, 1]$ 범위로 만든다. 예: 2분음표($d = 4$)와 8분음표($d = 1$)의 차이는 $3/4 = 0.75$, 같은 duration이면 0. $w_d = 0.3$은 duration 차이가 pitch 관계보다 덜 중요하다는 가정을 반영한다.
- **계수 미튜닝:** w_o, w_d는 경험적으로 설정되었으며 grid search를 수행하지 않았다. 향후 과제로 이 계수들을 체계적으로 최적화할 여지가 있다.

각 함수의 출력 형태:

- `tonnetz_note_distance`: Tonnetz 격자 거리(정수) + 옥타브(실수) + duration(실수) = **실수**
- `voice_leading_note_distance`: 반음 차이(정수) + duration(실수) = **실수** (pitch 성분만으로는 정수이나 duration 항 때문에 실수가 됨)
- `dft_note_distance`: DFT L_2 거리(실수) + 옥타브 + duration = **실수**

DFT 계산 예시. C4 ($p = 60$)와 E4 ($p = 64$)를 비교한다. pitch class는 각각 0 (C)과 4 (E)이다. 12차원 indicator vector $(1, 0, 0, \ldots, 0)$과 $(0, 0, 0, 0, 1, 0, \ldots, 0)$에 DFT를 적용하면 magnitude 벡터 $|\hat{f}_k|$ ($k = 1, \ldots, 6$)을 얻는다. 이 두 벡터 사이의 L_2 거리가 DFT 기본 거리이며, 여기에 옥타브 항($|5 - 5| \times 0.5 = 0$)과 duration 항을 더한다.

4.1 Experiment 1 — Distance Function Baseline 비교

네 종류의 거리 함수 각각으로 사전 계산한 중첩행렬을 로드하여 Algorithm 1을 실행한다. 각 거리 함수에서 발견되는 cycle의 수도 함께 보고한다.

거리 함수	발견 cycle 수	JS Divergence (mean ± std)	Note Coverage	생성 시간 (ms)
frequency (baseline)	43	0.0753 ± 0.0033	0.991	31.2
Tonnetz	46	0.0398 ± 0.0031	1.000	38.9
voice-leading	22	0.0891 ± 0.0048	1.000	22.2
DFT	20	0.0511 ± 0.0029	1.000	26.3

해석 1 — Tonnetz가 가장 우수. Tonnetz 거리 함수는 baseline(frequency)에 비해 JS divergence를 $0.0753 \to 0.0398$로 **약** 47% **감소**시켰다. 두 조건의 표준편차가 각각 0.0033, 0.0031로 매우 작으므로, 이 차이는 통계적으로 명확한 개선이다(자세한 분석은 3.3절).

해석 2 — 거리 함수가 위상 구조 자체를 바꾼다. 동일한 hibari 데이터에서 거리 함수만 교체했을 뿐인데 발견되는 cycle 수가 $20 \sim 46$으로 크게 달라졌다. 이는 "거리 함수의 선택이 곧 어떤 음악적 구조를 '동치'로 간주할 것인가를 정의한다"는 음악이론적 관점과 일치한다. Tonnetz는 완전 5도 / 장3도 / 단3도 관계의 pitch class 쌍을 가깝게 배치하므로, 이러한 관계를 공유하는 note들이 한 cycle에 더 자주 모이게 된다.

해석 3 — Note Coverage는 대부분의 설정에서 포화. 네 거리 함수 모두 note coverage가 $0.99 \sim 1.00$이므로, "원곡의 모든 note 종류가 생성곡에 최소 한 번 등장"하는 기본 요구는 모두 만족된다. 따라서 품질의 주된 차이는 "같은 note pool을 얼마나 자연스러운 비율로 섞는가"에서 발생한다.

4.2 Experiment 2 — Cycle Subset Ablation

거리 함수를 Tonnetz로 고정하고, cycle 수 K를 변화시켜 "더 많은 cycle = 더 좋은 생성인가?"를 검증한다. $K = 10$과 $K = 17$은 전체 46개 중 **cycle label 번호 순서대로 처음** K**개**를 취한 prefix subset이다 (cycle label은 persistence 계산 시 문자열 정렬 순으로 부여됨). 이 순서는 greedy 최적화가 아니라 단순 prefix이며, 이는 "cycle

수 자체의 효과"를 분리하기 위한 의도적 설계이다.

설정	K	JS Divergence	KL Divergence	Note Coverage	생성 시간 (ms)
Tonnetz, $K=10$	10	0.0991 ± 0.0038	0.556 ± 0.035	0.980	24.4
Tonnetz, $K=17$	17	0.0740 ± 0.0038	0.550 ± 0.344	0.996	26.3
Tonnetz, $K=46$ (full)	46	0.0397 ± 0.0025	0.172 ± 0.013	1.000	40.8

해석 4 — Cycle이 많을수록 JS가 단조 감소. K가 $10 \to 17 \to 46$으로 늘어남에 따라 JS divergence는 $0.099 \to 0.074 \to 0.040$으로 단조 감소하였다. 이는 "위상 구조가 더 풍부하게 드러날수록 생성곡의 음 사용 분포가 원곡에 더 근접한다"는 본 연구의 핵심 가설을 뒷받침한다.

해석 5 — 한계 효용의 감소. K 증가에 따른 JS 감소폭을 보면:

- $K=10 \to 17$: 개선 $\Delta = 0.025$
- $K=17 \to 46$: 개선 $\Delta = 0.034$

cycle 수가 거의 세 배($17 \to 46$)가 된 것에 비해 개선 폭은 $K=10 \to 17$(7개 추가)과 크게 차이나지 않는다. 즉 뒤쪽 cycle들은 이미 어느 정도 포화된 구조를 재확인하는 수준의 기여를 한다. 이는 2.7절에서 논의한 greedy forward selection으로 "소수의 cycle로도 90% 보존"이 가능하다는 관찰과 일관된다.

해석 6 — KL 분산의 불안정성. $K=17$ 설정에서 KL divergence의 표준편차가 0.344로 유난히 크다. 이는 일부 trial에서 KL이 1.55까지 튀는 경우가 있었기 때문이며 (원본 JSON은 `docs/step3_data/step3_results.json`의 `experiment_2_ablations.subset_K17.kl_divergence.max` 필드에서 확인 가능), "$\log(P/Q)$가 $Q \to 0$에서 발산하는" KL의 구조적 불안정성에서 기인한다. JS divergence는 동일 trial들에서 $0.064 \sim 0.079$ 범위로 안정되어 있어, JS가 더 안정적인 평가 지표임을 재확인한다 (2.6절의 "대칭화와 유계성" 논의와 일관).

4.3 통계적 유의성 분석

두 baseline 비교 (frequency vs Tonnetz) 의 차이가 통계적으로 유의한지 확인한다. 두 표본의 평균을 비교하는 표준적인 방법은 Student t-test이지만, Student의 고전적 t-test는 "두 집단의 모분산이 같다"는 강한 가정을 필요로 한다. 본 실험에서 두 조건의 표본표준편차는 $s_1 = 0.0033$, $s_2 = 0.0031$으로 매우 비슷하지만 완전히 같지는 않으며, 모분산이 같다는 사전 근거도 없다. 따라서 등분산 가정을 요구하지 않는 **Welch's t-test**를 사용한다. Welch는 "모평균과 모분산을 모를 때 표본평균과 표본분산만으로 검정"이 가능하며, 자유도를 Welch-Satterthwaite 근사로 계산한다.

데이터.

- Frequency: $\bar{x}_1 = 0.0753$, $s_1 = 0.0033$, $n_1 = 20$
- Tonnetz: $\bar{x}_2 = 0.0398$, $s_2 = 0.0031$, $n_2 = 20$

Welch t 통계량 은 다음과 같이 정의된다:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

수치를 대입하면:

$$t = \frac{0.0753 - 0.0398}{\sqrt{\frac{0.0033^2}{20} + \frac{0.0031^2}{20}}} = \frac{0.0355}{\sqrt{1.025 \times 10^{-6}}} \approx 35.1$$

Welch-Satterthwaite 자유도. Welch 검정에서 t 통계량은 정확한 Student 분포를 따르지 않고, 다음 식으로 근사된 자유도 ν의 t-분포를 따른다:

$$\nu \approx \frac{(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2})^2}{\frac{(s_1^2/n_1)^2}{n_1 - 1} + \frac{(s_2^2/n_2)^2}{n_2 - 1}}$$

$n_1 = n_2 = 20$이고 두 표본분산이 거의 같으므로 ν는 $\approx n_1 + n_2 - 2 = 38$에 매우 가깝게 나온다. 수치 계산 결과 $\nu \approx 37.9$이며, 반올림하여 $\nu = 38$로 사용한다.

자유도 $\nu = 38$에서 양측 임계값은 $t_{0.001,\,38} \approx 3.56$이므로, $|t| = 35.1 \gg 3.56$이며 $p < 10^{-20}$이다. 따라서 **Tonnetz가 frequency보다 JS divergence를 낮춘 것은 극도로 통계적으로 유의**하다.

효과 크기 (Cohen's d). p-값만으로는 "차이가 실질적으로 얼마나 큰가"를 알 수 없으므로, 표본평균 차를 표본표준편차로 정규화한 Cohen's d를 함께 보고한다:

$$d = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{(s_1^2 + s_2^2)/2}}$$

$$d = \frac{0.0355}{\sqrt{(0.0033^2 + 0.0031^2)/2}} \approx 11.1$$

Cohen의 관례상 $d > 0.8$이 "큰 효과"인데 $d \approx 11$은 비교할 수 없는 초대형 효과이다. 두 분포가 실질적으로 분리되어 있음을 의미한다.

4.3a Experiment 2.5 — Continuous Overlap Matrix 실험

본 절은 2.5절에서 정의한 **연속값 중첩행렬** $O_{\text{cont}} \in [0,1]^{T \times K}$가 이진 중첩행렬 $O \in \{0,1\}^{T \times K}$ 대비 어떤 영향을 주는지를 정량적으로 검증한다. 거리 함수는 모든 설정에서 Tonnetz로 고정한다 (§4.1 결과상 가장 강한 거리 함수). **본 실험은 Algorithm 1에 대해서만 수행하였다.** Algorithm 2(DL)에 continuous overlap을 적용하는 실험은 §4.5에서 별도로 다룬다.

실험 설계

cycle별 시점 활성도 $a_{c,t}$는 두 가지 방식으로 계산할 수 있다.

이진 (binary): 단순 OR 연산이다. $V(c)$에 속하는 note가 시점 t에 하나라도 활성이면 $a_{c,t} = 1$, 그렇지 않으면 0이다.

연속값 (continuous): cycle을 구성하는 note 중 얼마나 많은 비율이 활성화되어 있는지를 $[0,1]$ 실수로 표현한다. 분수 형태가 아니라 단일 라인으로 쓰면:

$$a_{c,t} = \Big(\sum_{n \in V(c)} w(n) \cdot \mathbb{1}[n \in A_t] \Big) \Big/ \Big(\sum_{n \in V(c)} w(n) \Big)$$

여기서 A_t는 시점 t에 활성인 note들의 집합, $w(n) = 1/N_{\text{cyc}}(n)$은 note n의 **희귀도 가중치**이며 $N_{\text{cyc}}(n)$은 note n이 등장하는 cycle의 개수이다. 적은 cycle에만 등장하는 희귀 note일수록 가중치 $w(n)$이 커져, 그 note가 활성화되면 $a_{c,t}$에 더 큰 기여를 한다.

연속값 활성도가 만들어진 후, 최종 overlap matrix를 만드는 방식에 따라 다시 두 가지 변형이 가능하다.

- **직접 사용 (direct)**: $O[t,c] = a_{c,t} \in [0,1]$
- **임계값 이진화 (threshold τ)**: $O[t,c] = \mathbb{1}[a_{c,t} \geq \tau]$, $\tau \in \{0.3, 0.5, 0.7\}$

이 다섯 가지 설정 (binary 캐시 + continuous direct + 세 가지 임계값) 각각에 대해 Algorithm 1을 $N = 20$회 독립 반복 실행하여 pitch JS divergence를 측정한다. 실험 러너는

`tda_pipeline/run_step3_continuous.py`이며 원본 결과는 `docs/step3_data/step3_continuous_results.json`에 저장되어 있다.

결과

설정	Density	JS Divergence (mean ± std)
(A) Binary (기존 캐시)	0.751	0.0387 ± 0.0027
(B) Continuous direct	0.264	0.0382 ± 0.0021
(C) Continuous → bin $\tau = 0.3$	0.373	0.0386 ± 0.0022
(C) Continuous → bin $\tau = 0.5$	0.201	0.0343 ± 0.0027
(C) Continuous → bin $\tau = 0.7$	0.077	0.0364 ± 0.0032

여기서 "Density"는 overlap matrix에서 활성으로 표시되는 셀의 평균 비율 ($\bar{O}$). Binary 캐시는 0.751로 매우 dense한 반면, continuous direct는 0.264로 훨씬 sparse하다 (희귀도 가중치가 평균값을 낮춘다).

해석

해석 7a — Continuous direct는 binary와 거의 동등. 단순히 활성도를 그대로 사용한 (B) 설정은 (A) 대비 평균 JS가 약간 낮고 (0.0382 vs 0.0387), 표준편차도 약간 작다 (0.0021 vs 0.0027). 차이는 통계적으로 유의하지 않으며, "연속값을 직접 쓰는 것 자체"는 큰 이득이 없다.

해석 7b — $\tau = 0.5$ 임계값이 최우수. 흥미롭게도 연속값을 만들어 놓고 다시 $\tau = 0.5$로 이진화한 설정이 평균 JS 0.0343로 가장 좋다. 이는 (A) baseline 대비 약 11.4% 개선이며, τ 변화에 대한 모양이 $0.3 \to 0.5$ 구간에서 단조 감소, $0.5 \to 0.7$에서 다시 증가하는 명확한 U자 패턴을 보인다.

이 결과의 음악적 해석은 다음과 같다. 기존 binary overlap (A)는 "한 cycle의 vertex 중 하나라도 활성이면 cycle 전체가 활성"으로 판정하므로 cycle 활성도가 과대평가되며 ($\bar{O} = 0.751$), Algorithm 1의 교집합 sampling이 너무 자주 호출되어 다양성이 떨어진다. 반대로 $\tau = 0.7$은 너무 까다로워서 cycle 활성 시점이 너무 드물어진다 ($\bar{O} = 0.077$). 그 사이의 $\tau = 0.5$가 최우수 결과를 만든다. 이 임계값의 의미는 "cycle의 희귀도 가중치 활성도가 50% 이상"이라는 것인데, 희귀도 가중치($w(n) = 1/N_{\text{cyc}}(n)$)가 note마다 다르므로 단순히 "vertex의 절반"이 아니라 "희귀 note의 활성 여부에 더 민감한 판정"이 된다. 즉 $\tau = 0.5$는 희귀 note가 활성화될 때 cycle을 더 쉽게 "살아 있음"으로 판정하여, 원곡에서 드물지만 중요한 음의 정보를 생성에 반영하는 효과가 있다.

해석 7c — Welch t-test (binary vs $\tau = 0.5$). 두 설정의 차이가 통계적으로 유의한지 검정한다.

- (A) Binary: $\bar{x}_A = 0.0387$, $s_A = 0.0027$, $n_A = 20$
- (C@τ=0.5): $\bar{x}_C = 0.0343$, $s_C = 0.0027$, $n_C = 20$

$$t = \frac{0.0387 - 0.0343}{\sqrt{0.0027^2/20 + 0.0027^2/20}} \approx 5.16$$

자유도 $\nu \approx 38$에서 $t_{0.001,38} \approx 3.56$이므로 $|t| = 5.16 > 3.56$, 즉 $p < 0.001$이다. Cohen의 효과 크기 $d \approx 1.63$ ($d > 0.8$ "큰 효과" 범주). 따라서 **continuous → $\tau = 0.5$ 이진화의 개선은 통계적으로 유의**하다.

이 결과는 본 연구의 향후 방향에 함의를 준다. 본 실험에서는 연속값 overlap matrix의 잠재력을 최소한만 사용했음에도 (단일 임계값 τ grid search 정도) 이미 약 11%의 추가 개선을 얻을 수 있었으며, 더 정교한 가중치 학습이나 soft activation을 받아들이는 Algorithm 2 변형을 통해 추가 향상이 가능할 것으로 기대된다.

4.4 Experiment 3 — DL 모델 비교 (참고)

왜 $N = 20$ 통계적 반복을 수행하지 않았는가

Algorithm 1은 한 번의 실행에 ~ 50 ms 만 걸리므로 $N = 20$ 반복이 수 초 안에 끝난다. 반면 Algorithm 2의 학습은 한 config 당 30 s ~ 3 min이 걸리므로, 동일한 수준의 반복 실험을 전체 grid에 대해 수행하려면 수

시간이 필요하다. 본 연구는 대신 다음 두 가지 선행 실험을 수행하였다.

1. **하이퍼파라미터 튜닝 (30 조합, 단일 run).** `run_tuning.py` 스크립트로 FC/LSTM/Transformer 각각에 대해 hidden dim $\in \{64, 128, 256\}$, learning rate $\in \{0.0005, 0.001, 0.003\}$, dropout $\in \{0.1, 0.3\}$, augmentation 배수 $\in \{5\times, 10\times\}$ 조합으로 grid search를 실행하였다. 이 튜닝에서 FC 모델이 top 10을 모두 차지했고, 최적 조합은 `FC / hidden=256 / lr=0.001 / dropout=0.3`으로 validation loss 0.282, pitch JS divergence 0.0015를 달성하였다. 튜닝 실험은 각 조합에 대해 1회만 수행되었으며, 동일 seed 하에서 결과가 결정적이므로 표준편차 정보는 없다.
2. **본 장에서 보고.** 아래 표는 튜닝에서 얻은 모델별 최우수 구성의 단일 run 결과를 그대로 가져온 것이다.

모델	Validation Loss	Pitch JS Divergence	생성 note 수
FC (hidden=256, lr=0.001, dropout=0.3)	0.282	0.0015	$3{,}754$
LSTM (2-layer, hidden=128)	0.385	0.0448	$3{,}753$
Transformer (2-layer, 4-head, $d_{\text{model}} = 128$)	0.676	0.0104	$3{,}753$

향후 $N = 5$ 반복 재검증이 Step 4에서 계획되어 있으나, 본 장에 DL 결과를 먼저 수록한 이유는 두 가지이다. 첫째, 거리 함수 비교 실험(3.1절)이 "Tonnetz가 Algorithm 1에서 우월하다"는 결론을 냈는데, **같은 결론이 DL 기반 Algorithm 2에서도 유효한지**를 동일 장에서 함께 논의해야 연구 전체의 일관성이 드러나기 때문이다(아래 해석 9). 둘째, DL 튜닝의 상위권이 FC 모델로 완전히 쏠린 것은 통계적 반복 없이도 확인되는 정성적 관측이며, 그 현상 자체가 hibari라는 곡의 고유한 성격에서 기인한다는 해석(해석 8)을 제시하기 위해서이다. Step 4의 재검증은 효과 크기의 수치적 안정성을 보강하는 용도이지, 현재 결론 자체를 뒤집기 위한 것이 아니다.

해석

해석 8 — FC가 가장 낮은 JS, 그리고 "Out of Noise"라는 맥락. 가장 단순한 FC 모델이 가장 낮은 JS divergence를 달성한 것은 얼핏 이상해 보인다. 통상적으로 시퀀스 모델(LSTM / Transformer)이 시간 문맥을 활용해 더 정교한 출력을 낼 것으로 기대되기 때문이다. 그러나 **hibari는 2009년 사카모토 류이치의 앨범 out of noise에 수록된 곡**이며, 이 앨범 자체가 "소음과 음악의 경계를 탐구한다"는 기획 의도로 제작되었다. 앨범의 많은 곡은 전통적 선율 진행을 의도적으로 회피하고, 각 음이 시간적 인과(causality)보다는 공간적 배치에 가까운 방식으로 놓인다.

이 미학적 맥락에서 보면 FC 모델의 우수한 결과는 역설이 아니라 **곡의 성격에 대한 실증**에 가깝다. FC는 시점 t의 note를 결정할 때 이전 시점 $t-1, t-2, \ldots$를 전혀 참조하지 않는, 다시 말해 "시간 맥락 없이 각 음을 독립적으로 배치하는" 모델이다. 이것이 LSTM/Transformer보다 원곡에 더 가까운 결과를 내었다는 것은, hibari가 선율적 인과보다는 음들의 비맥락적 배치로 구성되어 있다는 원곡의 작곡적 특성을 모델 선택이 간접적으로 드러낸 셈이다. 즉 FC가 "의미 없이 친 것 같은" 분포가 실제로 원곡과 일치했던 것은, 원곡 자체가 그러한 방식으로 설계되었기 때문이라는 해석이 가능하다.

해석 9 — Tonnetz + FC 조합의 최고 성능. Tonnetz hybrid distance ($\alpha = 0.5$) 로 사전 계산한 중첩행렬을 FC 모델에 입력했을 때, 선행 실험에서 pitch JS divergence ≈ 0.003을 기록하였다 (기준 frequency + FC의 0.053 대비 약 18**배 개선**). 이것이 본 연구 전체에서 관측된 최우수 결과이며, Experiment 1의 "Tonnetz가 우수한 구조적 사전"이라는 결론이 DL 기반 생성에서도 유효함을 시사한다.

4.4a Experiment 4 — Continuous overlap + Algorithm 2 (FC)

§4.3a에서 Algorithm 1에 대해 continuous overlap $\rightarrow$ $\tau = 0.5$ 이진화가 11% 개선을 주었다. 본 절은 동일한 아이디어를 Algorithm 2의 FC 모델에 적용한다 — 즉 continuous activation matrix를 FC의 학습 입력으로 직접 사용했을 때 얼마나 더 큰 효과가 있는가를 정량화한다. (본 실험은 §7.1에서 "개선 F"로 제안된 후 구현된 것이다. A~E까지의 개선이 선행되었으며, F는 continuous overlap을 DL에 적용하는 여섯 번째 개선이다.) 실험 러너는 `tda_pipeline/run_improvement_F.py`, 결과는 `docs/step3_data/step_improvementF_results.json`

에 저장되어 있다.

실험 설계

FC 모델 (§3.4 최우수 아키텍처: 2-layer, hidden 128, dropout 0.3) 을 고정하고, **학습 입력 데이터만** 두 가지로 바꾸어 비교한다.

F-bin (baseline): 기존 §3.4 와 동일. `build_activation_matrix(continuous=False)` 로 얻은 이진 활성행렬 $X_{\text{bin}} \in \{0,1\}^{T \times K}$ 를 FC 의 입력으로 사용.

F-cont: §4.3a의 "continuous direct"와 같은 방식, 즉 희귀도 가중치가 적용된 연속값 활성행렬 $X_{\text{cont}} \in [0,1]^{T \times K}$ 를 FC의 학습 입력으로 직접 사용. 구체적으로 $X_{\text{cont}}[t,c] = a_{c,t}$ (§4.3a 에서 정의한 $a_{c,t}$, $w(n) = 1/N_{\text{cyc}}(n)$ 의 희귀도 가중치). 모든 다른 조건 — 모델 아키텍처, learning rate, epochs, batch size, train/valid split — 는 F-bin 과 완전히 동일.

각 설정에 대해 서로 다른 torch seed $(8100, 8107, 8114, 8121, 8128)$ 로 $N = 5$ 회 학습 + 생성 + JS 측정.

결과

설정	JS (mean ± std)	best	validation loss (mean)	coverage
F-bin (기존 §3.4)	0.0014 ± 0.0010	0.0006	0.0702	0.948
F-cont ★	0.0006 ± 0.0005	0.0003	0.0312	0.991

핵심 발견 1: JS divergence 약 2.3배 감소. F-cont 의 평균 JS 0.0006 는 F-bin 의 0.0014 대비 약 57% 감소이다. §3.1 에서 distance function 교체 (frequency $\rightarrow$ Tonnetz) 가 47% 감소, §3.3a 에서 $\tau = 0.5$ 이진화가 추가 11% 감소를 주었다면, 본 개선 F 는 그 위에 또 한 단계 개선을 쌓아 $0.0014 \rightarrow 0.0006$ 을 달성한다.

핵심 발견 2: 분산도 2배 감소. F-bin 의 표준편차 0.0010 대비 F-cont 는 0.0005. 즉 seed 별 결과의 일관성도 개선되었다.

핵심 발견 3: Validation loss 도 2.3배 낮음 ($0.070 \rightarrow 0.031$). 이는 모델이 학습 자체를 더 잘 한다는 뜻으로, 연속값 표현이 이진화 대비 학습 신호가 더 풍부하고 gradient landscape 이 더 부드러움을 시사한다.

핵심 발견 4: Note coverage 가 $0.948 \rightarrow 0.991$ 로 대폭 향상. F-bin 에서는 일부 rare note 가 생성되지 않는 경우가 있었지만, F-cont 에서는 거의 전체 23개 note 가 생성된다. 이는 희귀도 가중치 ($w(n) = 1/N_{\text{cyc}}(n)$) 가 의도한 효과 — 희귀 note 에 더 큰 학습 신호 — 가 실제로 작동함을 뒷받침한다.

왜 이렇게 큰 개선이 나오는가 — 음악적 해석

이진화는 "있다/없다" 만 말하지만, 연속값은 "얼마나 확신하는가" 를 말한다. 이진 $X_{\text{bin}}[t,c]$ 는 "시점 t 에 cycle c 의 vertex 중 하나라도 울리면 1"이다. 이것은 cycle c 의 활성 여부에 대해 이진 판정만 내린다. 반면 연속값 $X_{\text{cont}}[t,c] = a_{c,t}$ 는 "cycle c 의 vertex 중 희귀도 가중치 기준으로 몇 퍼센트가 활성인가" 를 $[0,1]$ 실수로 표현한다. 즉 "이 시점에 이 cycle 이 어떤 종류의 note 조합으로 부분적으로 활성화되어 있는가" 라는 훨씬 풍부한 정보를 전달한다.

FC 모델 입장에서 이 차이는 결정적이다. 이진 입력에서 한 cycle 의 활성은 "on/off" 두 상태뿐이지만, 연속 입력에서는 "오직 common note 2 개만 활성 (낮은 값)" vs "rare note 포함 전체가 활성 (높은 값)" 같은 구별이 가능해진다. FC 가 이 구별을 학습하여 output note 분포를 조정할 수 있기 때문에, 원곡의 pitch 분포에 더 가까운 결과가 나온다.

기존 모든 결과와의 통합 비교

실험	설정	JS divergence	출처
§4.1 Algo 1	frequency baseline	0.0753	§4.1
§4.1 Algo 1	Tonnetz hybrid	0.0398	§4.1

§4.3a Algo 1	Tonnetz + continuous $\tau = 0.5$	0.0343	§4.3a
§7.1.6 Algo 1	P4 + C (module-level)	0.0590	§7.1
§7.1.8 Algo 1	P4 + C, best trial (seed 9302)	0.0258	§7.1
§4.4 Algo 2 FC	Tonnetz binary	0.0014	§4.4
§4.4a Algo 2 FC	**Tonnetz continuous**	0.0006 ± 0.0005	**본 절 ★**

> **§7.1.6 / §7.1.8 항목 요약 (세부 내용은 §7.1 참조).** §7.1은 hibari의 32-timestep 모듈 구조를 직접 활용하는 **모듈 단위 생성** 실험이다. **P4** 전략은 원곡 첫 모듈($t \in [0, 32)$)을 prototype으로 사용하고, **C** 전략(best-of-k selection)을 결합하여 JS를 최소화한다. §7.1.6 결과($N = 10$ 평균): JS 0.0590. §7.1.8에서 최적 seed(9302)에서는 JS 0.0258을 달성하였다. 이는 full-song Algorithm 1의 0.0398보다 낮은 값으로, 모듈 구조 활용이 유효한 개선임을 보여준다. 다만 Algorithm 2 (DL)의 0.0006보다는 크며, 이 대비가 "생성 방식마다 최적 적용 범위가 다르다"는 해석을 지지한다.

§4.4a는 본 연구의 full-song 생성에서 관측된 최저 JS divergence 이며, F-bin 대비 2.3배 우수하다. 이론적 최댓값 $\log 2 \approx 0.693$ 의 약 0.09% 에 해당한다.

한계 및 후속 과제

1. **학습 시 분산이 충분히 측정되었는가**: $N = 5$ 는 §3.1 의 $N = 20$ 에 비해 적다. FC 학습이 빠르므로 $N = 20$ 재확장이 비교적 쉽게 가능하다.
2. **LSTM / Transformer 에도 F-cont 적용**: 본 절은 FC 만 다루었다. §3.4 에서 관찰한 "FC > LSTM/Transformer" 패턴이 continuous 입력에서도 유지되는지 검증해야 한다.
3. **Continuous + module-local (P4)**: §7.1 의 P4 + C 와 개선 F 를 결합하여 "module-local continuous activation 을 FC 에 입력" 하는 실험도 의미가 있다.

4.5 종합 논의

(1) 음악이론적 거리 함수의 중요성. Experiment 1의 결과는 "빈도 기반 거리(frequency)는 기본 선택일 뿐, Tonnetz처럼 음악이론적 구조를 반영한 거리가 훨씬 더 좋은 위상적 표현을 만든다"는 본 연구의 가설을 강하게 지지한다. frequency → Tonnetz 전환만으로 JS divergence가 47% 감소했다.

(2) Cycle 수의 효과는 단조 증가이나 점진적. Experiment 2는 "cycle 수가 많을수록 좋다"는 기대를 확인하였으나, 한계 효용이 감소함을 보였다. 이는 K를 무작정 늘리기보다 greedy forward selection (2.7절) 으로 소수의 핵심 cycle을 고르는 전략이 실용적임을 시사한다.

(3) 통계적 엄밀성. 각 설정에서 $N = 20$ 반복을 통해 주요 baseline 비교의 Welch t 값이 > 35으로 매우 유의했다. 이는 본 연구의 결론이 random seed에 의존하는 artifact가 아님을 보장한다.

(4) 곡의 맥락과 모델 선택. FC가 시퀀스 모델을 능가한 해석 8의 관찰은, 모델의 성능이 단순히 "표현력이 높을수록 좋다"는 법칙을 따르지 않고 **원곡의 미학적 설계와 공명하는 모델**이 가장 좋은 결과를 낸다는 것을 보여준다. 이는 본 연구가 다른 곡(예: 전통적 선율 진행이 뚜렷한 클래식 작품)으로 확장될 때 시퀀스 모델의 우위가 뒤바뀔 수 있음을 암시한다.

(5) 향후 과제.

1. DL 모델 comparison을 $N = 5$ 이상 반복으로 재검증 (효과 크기 수치 안정화)
2. Ablation에서 prefix subset 대신 greedy selected subset으로 재실행하여 "어떤 cycle이 선택되는지"의 효과 분리
3. Tonnetz hybrid의 α 값 ($0.0, 0.3, 0.5, 0.7, 1.0$) 에 대한 grid search
4. 다른 곡(Ravel, Debussy, 또는 out of noise 앨범의 다른 곡) 으로의 일반화 검증 — 해석 8의 "곡의 미학이 모델 선택을 결정한다"는 가설 검증

4.6 곡 고유 구조 분석 — hibari 의 수학적 불변량

본 절은 hibari 가 가지는 수학적 고유 성질을 분석하고, 이 성질들이 본 연구의 실험 결과와 어떻게 연결되는지를 서술한다. 비교 대상으로 사카모토의 다른 곡인 solari 와 aqua 를 함께 분석한다.

4.6.1 Deep Scale Property — hibari 의 pitch class 집합이 갖는 대수적 고유성

hibari 가 사용하는 7개 pitch class 는 $\{0,2,4,5,7,9,11\} \subset \mathbb{Z}/12\mathbb{Z}$ 이다 (C major / A natural minor scale). 이 7개 pitch class 집합 전체의 **interval vector** (§2.10 정의 2.12 참조)는 $[2,5,4,3,6,1]$ 이다. 여기서 k 번째 성분은 "집합 안에서 interval class k 에 해당하는 쌍의 수"이다. interval class는 1~6까지만 존재한다 (7 반음 이상은 옥타브 대칭에 의해 $12-k$ 와 동치이므로). 따라서 벡터의 길이는 7이 아니라 **6**이다.

이 벡터의 6개 성분이 **모두 다른 수**이다 ($\{1,2,3,4,5,6\}$ 의 순열). 이것을 **deep scale property** 라 한다 (Gamer & Wilson, 2003). 이 성질을 갖는 7-note subset 은 $\binom{12}{7}=792$ 개 중 **diatonic scale 류 (장/단음계, 교회 선법) 뿐**이다. 즉 hibari 가 7개 PC 를 선택한 것은 임의가 아니라, 12음 체계에서 **각 음정 클래스가 고르게 (그러면서도 모두 다른 횟수로) 등장하는 유일한 부분집합**을 선택한 것이다.

또한 7개 PC 사이의 간격 패턴은 $[2,2,1,2,2,2,1]$ 로, 오직 $\{1,2\}$ 두 종류의 간격만으로 구성된다. 이것은 **maximal evenness** — 12개 칸 위에 7개 점을 가능한 한 균등하게 배치한 상태 — 를 의미한다 (Clough & Douthett, 1991). deep scale 과 maximal evenness 는 모두 diatonic scale 의 고유 성질이다.

solari 와 aqua 는 12개 PC 모두를 사용하므로 이 성질이 적용되지 않는다.

4.6.2 근균등 Pitch 분포 — Pitch Entropy

곡	사용 pitch 수	정규화 pitch entropy	해석
hibari	17	0.974	거의 완전 균등
solari	34	0.905	덜 균등
aqua	51	0.891	가장 치우침

pitch entropy는 곡 안에서 사용된 모든 pitch의 빈도 분포에 대한 **Shannon entropy**를 계산하고, 이론적 최댓값으로 나눠 정규화한 것이다. Shannon entropy $H=-\sum_i p_i \log_2 p_i$ 는 분포의 "불확실성" 또는 "균등함"을 측정하며, 모든 결과가 동일 확률일 때 최대이다. 정규화는 $H/\log_2(\mathrm{uniquepitchcount})$ 로, 1.0 이면 모든 pitch가 완전히 동일한 빈도로 등장하고, 0 에 가까우면 한두 개 pitch가 지배적이다.

hibari 의 0.974 는 **"모든 pitch 를 거의 같은 빈도로 사용"**한다는 뜻이며, 전통 조성 음악에서는 매우 드문 수치이다 (으뜸음이 지배적인 것이 보통). 이 성질은 **§3.4 의 "FC 모델 우위"를 수학적으로 설명**한다. pitch 분포가 거의 균등하면, 시간 순서를 무시하고 그 분포에서 독립적으로 뽑는 것 (FC 의 행동) 이 이미 원곡의 분포에 가깝다. 반면 solari 같이 특정 pitch 가 더 자주 나오는 곡에서는 시간 맥락 (Transformer) 이 그 편향을 학습해야 하므로 FC 가 불리하다.

4.6.3 Tonnetz 구별력과 Pitch Class 수의 관계

hibari 의 7개 PC 는 Tonnetz 위에서 **하나의 연결 성분**을 이루며, 평균 degree 가 $3.71/6=62\%$ 이다. Tonnetz 이웃 관계는 ± 3 (단3도), ± 4 (장3도), ± 7 (완전5도) 의 세 가지 방향이며, 각 방향이 양쪽으로 작용하므로 최대 6 개의 이웃이 가능하다.

예를 들어 C(0) 의 이웃을 계산하면:

관계	$+$ 방향	$-$ 방향	hibari 에 있는?
단3도 (± 3)	D#(3)	A(9)	A 만
장3도 (± 4)	E(4)	G#(8)	E 만

완전5도 (± 7)	G(7)	F(5)	둘 다

여기서 $0-7=-7=5 \pmod{12}=F$ 이다. 즉 C 에서 완전5도 **아래로** 내려가면 F 에 도달한다 (동시에, 완전4도 위로 올라가는 것과 같다). 따라서 C 의 Tonnetz 이웃은 $\{E, F, G, A\}$의 **4개** 이다.

왜 이것이 중요한가 — Tonnetz 그래프의 지름(diameter).

12개 PC **전부**를 사용하는 곡 (solari, aqua) 에서는 **어떤 두 PC 든 Tonnetz 거리가** ≤ 2 이다. 이유: 임의의 PC 에서 한 발짝 ($\pm 3, \pm 4, \pm 7$) 으로 도달 가능한 PC 가 6개이고, 나머지 5개 ($12-1-6=5$) 는 모두 두 발짝 안에 도달 가능하다. 예를 들어 C 에서 한 발짝에 $\{3,4,5,7,8,9\}$ 를 다 거치면, 거기서 한 발짝 더 가면 남은 $\{1,2,6,10,11\}$ 에 모두 도달한다 (예: $1=4-3,\ 2=5-3,\ 6=3+3,\ 10=7+3,\ 11=4+7$). $\mathbb{Z}/12\mathbb{Z}$ 의 대칭성으로 모든 PC 에서 동일. 따라서 12-PC Tonnetz 그래프의 지름은 2 이다.

지름이 2 라는 것은 **"가까운 음"과 "먼 음"을 구별할 여지가 거의 없다**는 뜻이다. 반면 hibari 의 7-PC 에서는 Tonnetz 거리가 $1 \sim 4$ 범위로 분포하여, 가까운 쌍 (예: C-G, 거리 1) 과 먼 쌍 (예: F-B, 거리 3 이상) 이 명확히 구별된다.

비유하면: 7명이 모여 사는 작은 마을에서 "누구와 누가 친한가"를 물으면 의미 있는 답이 나오지만, 12명이 모두 이웃인 세계에서는 "다 친하다"에 가까운 답이 나오는 것과 같다. 이것이 **hibari 에서 Tonnetz 가 최적 거리 함수인** 구조적 근거이다.

이 예측은 §7.2의 solari 실험에서 직접 검증되었다. solari 는 12-PC 반음계적 곡으로 Tonnetz 그래프의 지름이 2 에 불과하다. 실험 결과 Algorithm 1 기준으로 frequency와 Tonnetz 가 JS 0.063으로 동등하게 나타났고, voice-leading 은 오히려 0.078로 가장 낮은 성능을 보였다. **즉 12-PC 곡에서는 Tonnetz 의 구별력 저하로 인해 frequency 와 Tonnetz 가 유사한 수준으로 수렴하며, 이 결과가 §4.1의 해석("hibari 에서 Tonnetz 우위는 7-PC 음악에 고유한 특성")을 실증적으로 뒷받침한다.** 자세한 §7.2 결과는 해당 절을 참조.

4.6.4 Phase Shifting — inst 1 과 inst 2 의 서로소 주기 구조

§5 Figure 7 에서 관찰한 "inst 1 은 쉼 없이 연속, inst 2 는 모듈마다 쉼이 있다"는 패턴을 더 정밀하게 분석하면, 쉼의 위치가 모듈마다 정확히 1칸씩 이동한다는 것이 발견된다. 원본 파이프라인 (solo 제거 후) 기준으로 inst 2 의 모듈별 쉼 위치를 조사하면:

```
module  1: rest at position  0
module  2: rest at position  1
module  3: rest at position  2
module  4: rest at position  3
  ...
module  k: rest at position  k-1
```

이것은 **대각선 패턴**이다. 32-timestep 모듈 안에서 쉼의 위치가 매 모듈마다 정확히 1칸씩 오른쪽으로 밀린다. 32개 모듈을 거치면 쉼이 모듈 내 모든 위치 ($0, 1, 2, \ldots, 31$) 를 정확히 한 번씩 방문한다.

이 구조는 미니멀리즘 작곡가 Steve Reich 가 Piano Phase (1967) 에서 사용한 **phase shifting** 기법과 수학적으로 동일하다. 같은 패턴을 연주하는 두 악기 중 하나가 아주 조금 느리게 진행하여, 같은 패턴이 점점 어긋나다가 한참 뒤에야 원래 정렬로 돌아오는 것이다.

hibari 에서 이 phase shifting 은 다음과 같이 수치화된다.

	inst 1	inst 2
모듈 주기	$M=32$ (쉼 없음)	$M+1=33$ (32 음 + 1 쉼)
반복 횟수	33 회	32 회
총 길이	$33 \times 32 = 1{,}056$	$32 \times 33 = 1{,}056$

두 주기가 32 와 33 으로 **연속 정수 = 항상 서로소**($\gcd(32,33)=1$) 이다. 이 서로소 관계 때문에:

1. **두 악기가 같은 총 길이를 채움**: $33 \times 32 = 32 \times 33 = 1{,}056$ timestep.
2. **두 악기의 "위상(phase)" 이 곡 전체에서 한 번도 동기화되지 않음**: 매 모듈마다 쉼의 위치가 1칸씩 밀리므로, 32개 모듈을 거쳐야 비로소 원래 위치로 돌아온다.

이 구조는 수학적으로 **Euclidean rhythm** (Bjorklund, 2003; Toussaint, 2005) 과도 연결된다. Euclidean rhythm 은 "n 비트 중 k 개를 가능한 한 균등하게 배치" 하는 알고리즘으로, 아프리카 전통 음악과 전자 음악에서 널리 사용된다. hibari 의 경우 "33 칸 중 1 칸을 비운다" 를 32 번 반복하면서 매번 1칸씩 이동하는 것이 Euclidean rhythm 의 가장 단순한 형태이다.

음악적 효과. 이 서로소 구조는 inst 2 의 쉼이 모듈 내 **모든 위치를 최대한 균등하게 방문**함을 보장한다. 쉼이 특정 위치에 몰리지 않으므로 "어떤 시점을 잘라 봐도 음악의 밀도가 일정" 하다. 이것은 §4.6.2 에서 관찰한 근균등 pitch entropy (0.974) 와 일관된 설계 원리이다 — **pitch 선택도 균등하고, 쉼 배치도 균등하다**. 두 악기가 서로소 주기로 배치되어 있다는 사실은, 단순한 겹치기가 아니라 **수론적으로 최적인 위상 분리**가 달성되어 있음을 시사한다.

4.6.5 Cycle 교차 밀도 — 77%

Tonnetz 기반으로 발견된 46개 H_1 cycle 중, 쌍별 교집합을 계산하면 $\binom{46}{2} = 1{,}035$ 쌍 가운데 797 쌍 (77%) 이 적어도 하나의 vertex 를 공유한다. 즉 cycle 들이 독립적으로 흩어져 있는 것이 아니라 **"그물처럼 촘촘히 엮여 있다"**.

이 높은 교차 밀도는 두 가지 실험 결과와 직접 연결된다.

(a) Greedy selection 의 효과. 46개 cycle 이 서로 많은 vertex 를 공유하므로, 소수의 cycle 만 선택해도 전체 note pool 의 대부분을 커버할 수 있다. 실제로 §2.7 에서 15개 cycle ($\sim 33\%$) 만으로 90% 보존도를 달성한 것이 이 구조적 중복성 덕분이다.

(b) Algorithm 1 의 교집합 규칙이 잘 작동하는 이유. Algorithm 1 은 "활성 cycle 들의 교집합"에서 음을 뽑는데 (§2.10 규칙 1), 만약 cycle 들이 vertex 를 거의 공유하지 않으면 교집합이 항상 빈 집합이 되어 이 규칙이 작동하지 않는다. 77% 의 쌍이 vertex 를 공유하므로 교집합이 비어있지 않을 확률이 높고, 교집합에서 뽑힌 음은 "여러 cycle 이 공통으로 중요하게 여기는 음"이라는 구조적 의미를 갖는다.